Software Requirements Specification

For

Academic Integrity Code Comparison System- COSC 4P02

Version 2.0

Prepared by : Anirudh Sharma, Atharv Adhyapak, Ayaan Salim Rathod, Hannah Thomas, Param Bhaveshkumar Patel, Shweta Chainsingh Dhiman, Thanmaya Govind Nair

4P02 Group number 8, Null Pointers

10-03-2026

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

Table of Contents

1. Introduction 1
1.1 Document Purpose 1
1.2 Document convention 1
1.3 Product Scope 2
1.4 References 3
2. Overall Description 3
2.1 Product Perspective 3
2.2 User Classes and Characteristics 3
2.3 Operating Environment 4
2.4 Design and Implementation Constraints 4
2.5 Assumptions and Dependencies 4
2.5.1 Assumptions (Things we are assuming are true, but not guaranteed) 4
2.5.2 Dependencies (external factors the project relies on) 4
3. System Features 5
3.1 Assignment Submission (Student Interface) 5
3.2 File Processing & Sanitization (The Anonymization Pipeline) 6
3.3 Similarity Analysis Engine 7
3.4 Instructor Dashboard & Visualization 8
3.5 Reporting & Comparison Tools 8
3.6 Administrative Tools 9
3.7 Data Retention & Privacy (The "30-Day Rule") 9
4. Data Requirements 10
4.1 Logical Data Model 10
4.2 Data Dictionary 10
4.2.1 Entity: Instructor 10
4.2.2 Entity: Assignment 11
4.2.3 Entity: Submission 12
4.2.4 Entity: StoredFile 13
4.2.5 Entity: ComparisonResult 13
4.2.6 Entity: Boilerplate 14
4.2.7 Entity: Course 14
4.3 Reports 15
4.4 Data Acquisition, Integrity, Retention, and Disposal 16
4.4.1 Data Acquisition (Filename Sanitization) 16
4.4.2 Data Retention (The "Keep Forever" Policy) 16
4.4.3 Data Disposal (The "30-Day Purge") 16
5. External Interface Requirements 16
5.1 User Interfaces 16
5.1.1 General UI Requirements 16

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

5.1.2 Instructor Dashboard 17
5.1.3 Student Submission Portal 18
5.2 Software Interfaces 18
5.2.1 Instructor Authentication 18
5.2.2 Database Interface 19
5.3 Hardware Interfaces 19
5.4 Communications Interfaces 19
5.4.1 Network Security 19
5.4.2 Notifications 19
6. Quality Attributes 19
6.1 Usability 19
6.2 Performance 20
6.3 Availability 20
6.4 Security 20
6.5 Portability & Installability 21
7. Other Requirements 21
7.1 Legal and Regulatory Compliance 21
7.2 Internationalization and Localization 21
8. Glossary 21
8.1 Acronyms and Abbreviations 21
8.2 Definitions 22
9. Analysis Models 24
9.1 Sequence Diagrams 24
9.1.1 Student Submission Sequence (No-Login Flow) 24
9.1.2 Instructor Analysis Sequence 25
10 State Machine Diagram 26
10.1 Submission Lifecycle State Machine 26
10.2 Analysis Lifecycle State Machine (Instructor) 27
10.3 Data Flow Diagrams (DFD) - Level 1 28
10.3.1 Student Submission & Privacy Pipeline 28
10.3.2 Instructor Analysis & Dashboard Workflow 28
10.4 System Architecture Diagram 29
10.4.1 Dockerized Microservices Architecture 29

Revision

Name	Date	Reason For Changes	Version
Requirements	10/03/2026	Added and removed requirements	2

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

1. Introduction

1.1 Document Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements for the **Academic Integrity Code Comparison System**. This document guides the development team in building a web-based platform that detects plagiarism in student source code submissions while strictly adhering to privacy and data protection regulations (Freedom of Information and Protection of Privacy Act **(FIPPA)**).

1.2 Document convention

This section describes the standards and conventions used throughout the Software Requirements Specification (SRS) for the Academic Integrity Code Comparison System. Adhering to these conventions ensures clarity, consistency, and traceability across the document.

Convention	**Description**	**Example**
Requirement ID	Every requirement is assigned a unique identifier based on its section and sequence.	REQ-3.1.1, REQ-5.1.2
Prioritization	Requirements are prioritized (though distinct priority levels like High/Medium/Low are implicit in the "Shall" statements, the ID order often implies flow).	Priority: High
Imperative Style	The keyword **"Shall"** indicates a mandatory requirement. **"Will"** indicates a statement of fact or future state. **"Should"** indicates a desirable but non-mandatory goal.	"The system **shall** validate the Access Key."
Stimulus/Response	Functional requirements are	**Stimulus:** User uploads file.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

	often described using a trigger event (Stimulus) followed by the system's reaction (Response).	**Response:** System sanitizes filename.
Data Dictionary	Data entities are defined in tables specifying the Element Name, Data Type, and Functional Description.	**Element:** StudentID **Type:** String (Encrypted)
Date & Time	Timestamps are formatted in **ISO 8601** to ensure sorting accuracy and avoid ambiguity.	YYYY-MM-DD HH:MM
User Interface	All UI elements, error messages, and system feedback are presented in **English**.	"Submission Successful"
Privacy Terminology	Clear distinction is made between **"PII"** (Personally Identifiable Information) and **"Anonymized Data"**.	**PII:** Student Name **Anonymized:** UUID-renamed file
Diagramming	**UML** (Unified Modeling Language) is used for Sequence and State diagrams. **DFD** (Data Flow Diagram) is used for data processing pipelines.	Sequence Diagram 9.1.1
Typography	**Bold** text is used for headlines, key terms, and UI buttons/labels.	Click **"Submit"**

1.3 Product Scope

The **Academic Integrity Code Comparison System** is a stateless submission and analysis platform designed for university computer science courses.

- **Objective:** To allow instructors to compare student code submissions against current class files and historical repositories to detect similarity.
- **Core Functionality:**

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

- Secure, anonymous student submissions using shared Access Keys (No-Login).
- Automated sanitization of code (stripping comments, metadata and boilerplate code).
- Internal similarity analysis engine (No external AI).
- Instructor dashboards for visualization and PDF reporting.

- **Privacy Focus:** The system ensures mandatory compliance with the **FIPPA of Ontario (R.S.O. 1990, c. F.31)** and related institutional policies by :
  - Implementing a data minimization approach (No Student Accounts).
  - Anonymizing and encrypting student data (PII).
  - Automatically purging Personal Identifiers after the statutory retention period, while retaining non-PII code for historical comparison.

1.4 References

1.[1]I. Sommerville, "Software Engineering: Principles and Practice," *Software Engineering Journal*, vol. 9, no. 5, p. 228, 1994, doi: https://doi.org/10.1049/sej.1994.0029.
2.[1]Accessibility for Ontarians with Disabilities Act, "The Act (AODA)," Accessibility for Ontarians with Disabilities Act (AODA), 2014. https://www.aoda.ca/the-act/
3.[1]Government of Ontario, "Accessibility for Ontarians with Disabilities Act, 2005, S.O. 2005, c. 11," *Ontario.ca*, Jul. 24, 2014. https://www.ontario.ca/laws/statute/05a11
4.[1]"Protection of Privacy," *Brock University*, 2024. https://brocku.ca/legal/fippa/ & https://www.ontario.ca/laws/statute/90f31

2. Overall Description

2.1 Product Perspective

This system is a standalone web application designed to be "housed on any server" via Docker containerization. It replaces manual plagiarism checks and ensures privacy compliance. It interacts with:

- **Users:** Via standard Web Browsers.
- **File Storage:** For retaining anonymized code archives.
- **Database:** For managing assignment configurations and keys.

2.2 User Classes and Characteristics

- **Student (Guest/Anonymous):**
  - **Goal:** Submit assignments and receive confirmation.
  - **Access:** Uses a shared Access Key. No permanent account.
  - **Characteristics:** Varies in technical skill; requires a highly intuitive, error-proof upload interface.
- **Instructor (Admin):**
  - **Goal:** Configure assignments, view similarity reports, and manage repositories.
  - **Access:** Authenticated via Username/Password.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

- **Characteristics:** Needs detailed analytics tools and the ability to drill down into code differences.

2.3 Operating Environment

- **Client:** Modern Web Browsers (Chrome, Edge, Firefox, Safari).
- **Server:** Any environment capable of running **Docker Containers** (e.g., Linux VM, Cloud Instance).
- **Database:** Relational Database (e.g., PostgreSQL) and File Object Storage.

2.4 Design and Implementation Constraints

1. **No External AI:** The comparison engine must use internal algorithms (e.g., tokenization). Sending code to external AI APIs (e.g., OpenAI) is strictly prohibited.
2. **Language Support:** The system must support C, C++, and Java at a minimum.
3. **Privacy & Anonymization:** Student identities must be decoupled from code files. Personal metadata must be automatically purged 30 days after the deadline.(FIPPA)
4. **No Student Accounts:** The system must operate without student user accounts to minimize data liability.(FIPPA)

2.5 Assumptions and Dependencies

2.5.1 Assumptions (Things we are assuming are true, but not guaranteed)

1. Users have internet access and a modern browser
    a. If not true, web-only access becomes a blocker.
2. University policy allows storing submissions for comparison
    a. The system assumes the institution permits retention of code submissions for integrity checking, including across offerings, subject to retention rules.
3. Instructors are authorized to select secondary repositories and upload secondary repositories
    a. Assumes instructors have rights to compare against historical offerings and defined repositories.
4. Submissions are in supported languages or identifiable formats
    a. Assumes files can be categorized as C/C++/Java (e.g., by extension or declared language).
    b. If students submit unusual build systems or mixed-language projects, analysis of quality may degrade unless explicitly handled.

2.5.2 Dependencies (external factors the project relies on)

1. Identity provider:
    a. Dependency on some mechanism to identify students/instructors (no authentication):
        i. Access Key.
        ii. Student Name, Student ID.
2. Server infrastructure availability:
    a. Needs a host machine, DNS (optional), and the ability to run a web service.
    b. If firewall/NAT prevents inbound access, internet-client requirements are harder to meet.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

3. Storage capacity:
    a. Repositories may grow across offerings depending on having enough disk/object storage.
4. Language parsing/tokenization libraries:
    a. If we use third-party parsers/tokenizers internally, our engine depends on their compatibility with C/C++/Java.
    b. If those libraries fail or don't support edge cases, analysis of accuracy suffers.
5. Legal/privacy interpretation:
    a. Our compliance behavior depends on an agreed interpretation of what constitutes personal info in:
        i. metadata (name, ID, email),
        ii. code comments (students may include names),
        iii. file paths and repository structure.
    b. If policy interpretation changes, requirements change (e.g., stricter anonymization, stricter retention limits).
6. Instructor-defined repositories availability
    a. As secondary repositories are external (e.g., imported archives), analysis depends on them being accessible, correctly formatted, and permitted to use.

3. System Features

This section outlines the specific functional requirements of the Academic Integrity Code Comparison System. The system is designed as a stateless submission portal with a backend analysis engine, prioritizing student privacy and "no-login" accessibility.

3.1 Assignment Submission (Student Interface)

This feature governs how students submit work without persistent user accounts, ensuring accessibility while preventing unauthorized uploads.

- **Priority: High**

Stimulus/Response Sequence:

1. **Stimulus:** Student navigates to the portal and enters the Assignment Access Key.
    a. **Response:** System validates the key; if valid, displays the Upload Form. If invalid or past deadline, system will display appropriate message
2. **Stimulus:** Student enters Name/ID, selects a file, and clicks "Submit."
    a. **Response:** System strips comments, renames the file to a UUID, encrypts metadata, and displays a "Submission Successful" receipt with a timestamp.

Functional Requirements:

- **REQ-3.1.1:** The system shall allow users to access the submission portal using a shared, alphanumeric **Access Key** provided by the instructor for each specific assignment.
- **REQ-3.1.2:** The system shall validate the Access Key before presenting the file upload interface.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

- **REQ-3.1.3:** The system shall require the user to manually enter **Student Metadata** (Student Name and Student ID) during the upload process.
- **REQ-3.1.4:** The system shall restrict students to a maximum of **three (3) submissions** per assignment.
- **REQ-3.1.5:** Upon a new submission matching an existing Student ID, the system shall automatically mark the previous submission as "Archived" and the new submission as "Active."
- **REQ-3.1.6:** The system shall strictly reject any upload attempts that occur after the instructor-defined deadline timestamp.

3.2 File Processing & Sanitization (The Anonymization Pipeline)

This feature ensures that all code is normalized and stripped of personal identifiers before storage or analysis.

- **Priority: High**

Stimulus/Response Sequence:

1. **Stimulus:** System receives a raw file upload.
    a. **Response:** System separates the file content from the filename/metadata.
    b. **Response:** System parses the code, removes all comments (// and /* */), and saves the sanitized version.
    c. **Response:** System deletes the original filename and assigns a random system ID (UUID).

Functional Requirements:

- **REQ-3.2.1:** The system shall accept individual source code files (e.g., .c, .cpp, .java) and compressed archives (.zip).
- **REQ-3.2.2:** The system shall automatically extract .zip archives and identify valid source code files for processing, discarding non-code assets.
- **REQ-3.2.3:** The system shall **strip all comments** (both single-line and block comments) from the source code immediately upon receipt to remove potential identifying information (e.g., @author tags).
- The system shall exclude **boilerplate code** from the submission.
- **REQ-3.2.4:** The system shall separate the Student Metadata (Name/ID) from the code content. The Metadata shall be stored in an encrypted table accessible only to the instructor, linked to the code via an anonymized Submission ID.
- **REQ-3.2.5 (Filename Sanitization):** The system shall discard the original filename provided by the student and rename the stored file using a unique, random system identifier (UUID) to prevent identity leakage via filenames.
- **REQ-3.2.6 (Sanitization Uniformity):** The system shall apply all code sanitization rules (stripping comments and boilerplate code as defined in REQ-3.2.3) to all source code files extracted from a **Secondary Repository** at the time of upload/import before storing them as StoredFile records.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

3.3 Similarity Analysis Engine

This feature describes the core logic for detecting plagiarism using internal algorithms.

- **Priority: High**

Stimulus/Response Sequence:

1. **Stimulus:** Instructor clicks "Run Analysis" -> “ Internal Analysis”
    a. **Response:** System retrieves all "Active" submissions
    b. **Response:** System subtracts "Boilerplate" code patterns from all files.
    c. **Response:** System compares every file against every other file and generates a Similarity Index.
2. **Stimulus:** Instructor clicks "Run Analysis" -> “ Compare Against Repositories”
    a. **Response:** System retrieves all "Active" submissions and selected Secondary Repositories.
    b. **Response:** System subtracts "Boilerplate" code patterns from all files.
    c. **Response:** System compares every file against every other file and generates a Similarity Index.

Functional Requirements:

- **REQ-3.3.1:** The system shall perform a similarity comparison between the "Active" submission and:
    - All other "Active" submissions in the current assignment.
    - Any submissions contained in **Secondary Repositories** (past course offerings) selected by the instructor.
- **REQ-3.3.2 (Boilerplate Exclusion):** The system shall exclude code segments that match the **Template/Boilerplate File** uploaded by the instructor, preventing false positives from starter code.
- **REQ-3.3.3:** The system shall calculate a **Similarity Index** (0-100%) for every pair of compared submissions.
- **REQ-3.3.4:** The system shall utilize internal algorithmic models (e.g., tokenization, string matching) for comparison. The system shall **not** transmit code to external AI services.
- **REQ-3.3.5:** The system shall identify **"Collusion Clusters"** (groups of 3 or more submissions) where the Similarity Index between all members exceeds a high threshold (e.g., >80%).
- **REQ-3.3.6 (Lexical Tokenization):** The system shall convert the raw source code of every submission into a standardized stream of tokens (Lexical Analysis), automatically discarding all whitespace, newline characters, and formatting styles.
- **REQ-3.3.7 (Identifier Normalization):** The system shall replace all user-defined variable and function names with generic placeholders (e.g., replacing int myCount = 0; with TYPE_INT IDENTIFIER ASSIGN LITERAL_NUM SEMICOLON) to ensure that simple "Find and Replace" renaming strategies do not lower the similarity score.
- **REQ-3.3.8 (Fingerprinting Algorithm):** The system shall generate a unique digital "Fingerprint" for each submission using the **Winnowing Algorithm**. This involves generating k-grams (overlapping sequences of code tokens), hashing them using a Rolling Hash (e.g., Rabin-Karp), and selecting a subset of hashes to represent the file.
- **REQ-3.3.9:** The system shall calculate a Similarity Index (0-100%) by counting the number of

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

matching hash fingerprints between two files and expressing it as a percentage of the total code volume.

- **REQ-3.3.10 (Language-Specific Parsing):** The system shall utilize distinct lexers for C, C++, and Java to ensure that language-specific syntax (such as preprocessor directives in C/C++ or annotations in Java) is correctly identified and normalized.
- **REQ-3.3.11**: The system shall restrict similarity comparisons to files of the same programming language (e.g., only comparing Java submissions against other Java files in the repository) to optimize performance and ensure analysis relevance.
- **REQ-3.3.12 (Quality Filtering of Matches):** The system shall apply additional filtering criteria to similarity matches, including minimum span length, minimum unique hash count, and minimum pair count thresholds to eliminate weak or generic matches.
- **REQ-3.3.13 (Filtered Similarity Scoring):** The system shall compute the final Similarity Index using only the retained matches that satisfy the defined quality thresholds.
- **REQ-3.3.14 (Match Confidence Classification):** The system shall assign a confidence level (e.g., High, Medium, Low) to each detected matching block based on its strength and characteristics.
- **REQ-3.3.15 (Match Confidence Metadata Exposure):** The system shall expose match confidence levels and span information for each detected block to the user interface for visualization.
- **REQ-3.3.16 (Balanced Hash Pairing):** The system shall limit matching occurrences of shared hashes between two submissions to the minimum frequency present in both files to prevent excessive highlighting.
- **REQ-3.3.17 (Match Block Merging):** The system shall merge adjacent or nearby matching segments within a defined character gap threshold into larger continuous blocks to improve readability of comparison results.

3.4 Instructor Dashboard & Visualization

This feature provides the instructor with high-level metrics to assess class-wide integrity trends.

- **Priority: Medium**

Functional Requirements:

- **REQ-3.4.1 (Distribution Graph):** The system shall generate a histogram or bar chart displaying the distribution of similarity scores across the class.
- **REQ-3.4.2:** The system shall allow filtering submissions by custom similarity index ranges (e.g., 81–100%).
- **REQ-3.4.3:** The system shall allow sorting similarity scores in ascending or descending order.
- **REQ-3.4.4 (Collusion Clusters):** The system shall display "Collusion Clusters" as grouped items in the dashboard.
- **REQ-3.4.5 (Similarity Heatmap)**: The system shall display a “Similarity Heatmap” showing pairwise similarity scores between all submissions.
- **REQ-3.4.6 (Dashboard)**: The system shall display summary metrics including:
    - Active Assignments

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

- Total submissions
- Number of repositories
- Repository library
- Archived assignments
- For each assignment:
    - Total Submissions
    - Total Files
    - # Avg lines of code
    - Latest Submission time stamp

3.5 Reporting & Comparison Tools

This feature allows the instructor to drill down into specific cases and generate evidence.

- **Priority:Medium**

Stimulus/Response Sequence:

1. **Stimulus:** Instructor selects a submission pair with a high similarity score.
    a. **Response:** System displays both files side-by-side.
2. **Stimulus:** Instructor clicks "Reveal Identity."
    a. **Response:** System decrypts the metadata and displays real student names.
3. **Stimulus:** Instructor clicks "Export PDF."
    a. **Response:** System generates a download link for the evidence report.

Functional Requirements:

- **REQ-3.5.1:** The system shall provide a **Side-by-Side Comparison View** for any selected pair of submissions.
- **REQ-3.5.2:** The Comparison View shall visually highlight the exact lines of code that match between the two files.
- **REQ-3.5.3:** The system shall display submissions anonymously (e.g., "Submission A") by default, but provide a **"Reveal Identity"** control for the instructor to view the associated Student Name and ID.
- **REQ-3.5.4:** The system shall generate a **Downloadable PDF Report** for a selected comparison, containing the similarity score, highlighted code segments.

3.6 Administrative Tools

This feature provides utility functions for managing assignment data.

- **Priority: Low**

Functional Requirements:

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

- **REQ-3.6.1 (Bulk Download):** The system shall allow the instructor to download all "Active" submissions for an assignment as a single compressed .zip file.
- **REQ-3.6.2:** The system shall allow the instructor to upload a **Boilerplate/Template File** at any time, triggering a re-analysis of similarity scores to exclude the provided template code.
- **REQ-3.6.3:** The system shall allow the instructor to upload and manage past repositories as a .zip file and upload it to the server.
- **REQ-3.6.4**: The system shall allow instructors to create new assignments.
- **REQ-3.6.5**: The system shall allow instructors to edit existing assignments.

3.7 Data Retention & Privacy (The "30-Day Rule")

This feature ensures compliance with privacy regulations regarding the storage of student data.

- **Priority: High**

Functional Requirements:

- **REQ-3.7.1:** The system shall execute an automated background process to permanently delete all **Student Metadata** (Name and Student ID) **30 days** after the assignment deadline.
- **REQ-3.7.2:** The system shall retain the anonymized source code in the **Permanent Repository** indefinitely for use as a Secondary Repository in future comparisons.
- **REQ-3.7.3:** The system shall ensure that once Metadata is deleted, the stored code cannot be traced back to the original student author by the system.

4. Data Requirements

This section describes the logical structure of the data processed by the system, including data entities, retention policies, and privacy constraints.

4.1 Logical Data Model

The system operates on a relational data model centered around the **Assignment** and **Submission** entities, managed by **Instructors**.

- **Instructor:** A registered faculty member with system access. They own Assignments.
- **Course:** Represents a specific offering (e.g., "COSC 4P02 Fall 2024"). Assignments are grouped under a Course to allows for "Previous Offering" comparisons.
- **Assignment:** Represents a specific coursework item created by an instructor. It acts as the container for all submissions and is linked to a Course.
- **Submission:** Represents a student's attempt to submit work. It contains the encrypted identity metadata and links to the stored source code files.
- **FileBlob (StoredFile):** The actual binary source code stored in the file system, along with its

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

pre-calculated "Winnowing" fingerprint hashes.
- **Boilerplate:** Instructor-provided code templates linked to an Assignment. These are excluded from analysis to prevent false positives.
- **ComparisonResult:** The output of the analysis engine. It links two submissions (Source and Target) and stores the calculated percentage of similarity between them.

4.2 Data Dictionary

The following table defines the specific data elements managed by the system.

4.2.1 Entity: Instructor

Data Element	Type	Description
InstructorID	UUID	Unique identifier for the instructor.
Username	String	The login username for the instructor.
PasswordHash	String	The secure, hashed password for authentication.
Email	String	The instructor's email address (used for system alerts, NOT for student communication).

4.2.2 Entity: Assignment

Data Element	Type	Description
AssignmentID	UUID	Unique identifier for the assignment.
CourseID	UUID	Foreign Key linking the assignment to a specific Course offering.
InstructorID	UUID	Foreign key linking to the

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

		Instructor who created it.
AccessKey	String (Alphanumeric)	A long, random string generated by the system (e.g., a1-8f92-x9p) shared with students.
Language	Enum	The programming language enforced for this assignment (e.g., JAVA, CPP, C).
Deadline	Timestamp	The exact date and time after which uploads are rejected.
Status	Enum	{Open, Closed, Archived}.

4.2.3 Entity: Submission

Data Element	**Type**	**Description**
SubmissionID	UUID	Unique identifier for the submission. Serves as the primary key for analysis.
AssignmentID	UUID	Foreign key linking to the parent Assignment.
StudentName	String (Encrypted)	The student's full name. Stored in an encrypted format. **NULL** after 30-day retention period.
StudentID	String (Encrypted)	The student's university ID number. **NULL** after 30-day retention period.
Timestamp	Timestamp	The exact time the upload completed.
SubmissionStatus	Enum	{Active, Archived}. Only Active submissions are used for similarity analysis.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

AttemptNumber	Integer	Tracks if this is the 1st, 2nd, or 3rd submission for this student.
IsFlagged	Boolean	Indicates if the instructor has manually flagged this submission for review.
ReviewNotes	Text	Private comments added by the instructor (not visible to students).

4.2.4 Entity: StoredFile

Data Element	Type	Description
FileID	UUID	Unique identifier for the specific file.
SubmissionID	UUID	Foreign key linking to the Submission.
StoragePath	String	The internal path to the file blob (e.g., /assignments/2024/fall/uuid_592.java). **Original filenames are discarded.**
FileHash	SHA-256	A cryptographic hash of the file content used to detect exact duplicates instantly.
ByteSize	Integer	The size of the file in bytes.
LineCount	Integer	The number of lines of code in the file (calculated after comment stripping).
FingerprintHash	Binary/Blob	The serialized Winnowing fingerprints (k-gram hashes) stored to optimize repeated comparisons without

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

		re-parsing source code.

4.2.5 Entity: ComparisonResult

This entity stores the output of the analysis engine, linking two submissions that share significant similarity.

Data Element	Type	Description
ResultID	UUID	Unique identifier for this comparison result.
SourceSubmissionID	UUID	Foreign Key linking to the primary Submission being analyzed.
TargetSubmissionID	UUID	Foreign Key linking to the matched Submission (from class or repository).
SimilarityScore	Integer	The similarity percentage (0-100).
MatchMetadata	JSON	Data structure identifying specific matching line numbers or token ranges for highlighting in the UI.

4.2.6 Entity: Boilerplate

This entity represents the template code provided by the instructor, which must be excluded from analysis.

Data Element	Type	Description
BoilerplateID	UUID	Unique identifier.
AssignmentID	UUID	Foreign Key linking to the Assignment.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

FilePath	String	Storage location of the template file.
FingerprintData	Binary	Pre-calculated hash signatures of the boilerplate code for rapid exclusion.

4.2.7 Entity: Course

This entity groups assignments into specific offerings (e.g., "COSC 4P02 Fall 2024"), allowing for historical comparison against "previous offerings."

Data Element	Type	Description
CourseID	UUID	Unique identifier for the course offering.
InstructorID	UUID	Foreign Key linking to the Instructor who teaches this course.
CourseCode	String	The official course code (e.g., "COSC 4P02").
Term	String	The academic term (e.g., "Fall 2024").
IsArchived	Boolean	Indicates if this is a past course offering used only for reference repositories.

4.3 Reports

The system generates the following reports based on the data above:

- **Similarity Analysis Report (PDF):** A static document generated on-demand containing the side-by-side comparison of two Submission entities.
- **Class Integrity Dashboard:** A real-time visualization aggregating Submission similarity scores.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

4.4 Data Acquisition, Integrity, Retention, and Disposal

This section outlines the protocols for managing student submissions from the point of entry to long-term storage and eventual anonymization.

4.4.1 Data Acquisition (Filename Sanitization)

Upon upload, the system shall **immediately discard** the original filename provided by the student. The file shall be renamed using a system-generated UUID before being saved to storage.

4.4.2 Data Retention (The "Keep Forever" Policy)

The system shall retain the **FileBlob** and **Submission** record indefinitely.

- **Archived Submissions:** Previous drafts (Archived status) are **not** deleted. They remain stored to provide a complete history of the student's work.

4.4.3 Data Disposal (The "30-Day Purge")

To comply with privacy requirements, the system includes an automated disposal routine:

- **Trigger:** 30 days after the Assignment deadline.
- **Action:** The system updates the Submission table to set StudentName and StudentID to NULL.
- **Result:** The code remains in the repository for future comparison, but it is no longer mathematically possible to link it back to the specific student.

5. External Interface Requirements

This section defines the interfaces between the system and its users, software components, and communication infrastructure.

5.1 User Interfaces

The system provides two distinct browser-based interfaces: the **Student Submission Portal** and the **Instructor Dashboard**.

5.1.1 General UI Requirements

- **REQ-5.1.1.1 (Browser Compatibility):** All user interfaces shall be fully functional on modern standard web browsers (Chrome, Edge, Firefox, Safari). Internet Explorer and legacy browsers are not supported.
- **REQ-5.1.1.2 (Accessibility):** The user interfaces shall comply with **AODA and WCAG 2.1 Level**

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

AA standards to ensure accessibility for users with disabilities (e.g., screen reader compatibility, keyboard navigation).

5.1.2 Instructor Dashboard

- **REQ-5.1.2.1 (Desktop Optimization):** The Instructor Dashboard (analysis, reporting, and configuration) shall be optimized for **Desktop and Laptop** screens (minimum resolution 1280x720). Mobile support for the dashboard is not required due to the complexity of code comparison views.
- **REQ-5.1.2.2:** The dashboard shall provide visual feedback (loading spinners, progress bars) for all analysis operations that take longer than 1 second.

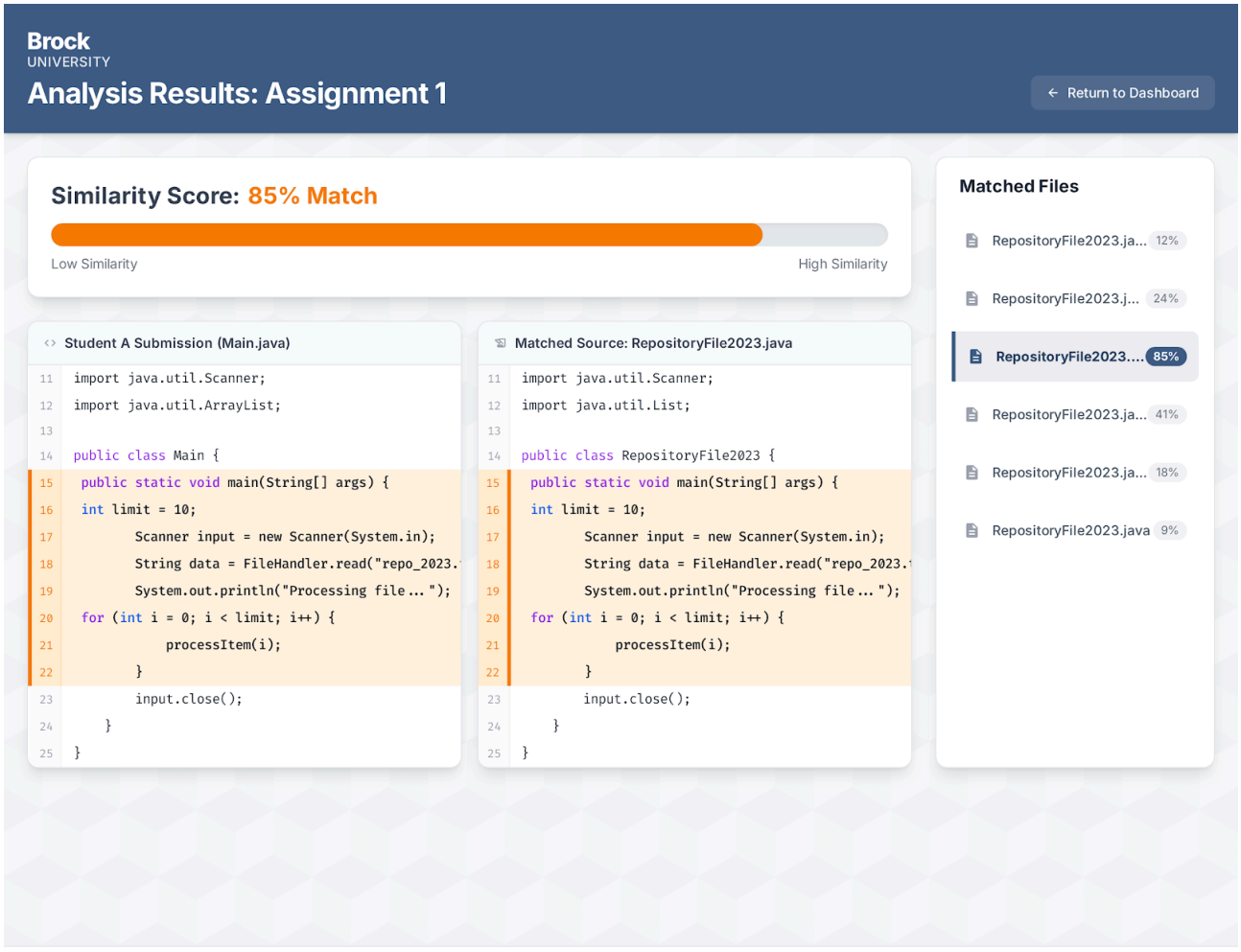


Figure 1: Instructor Similarity Comparison Interface

5.1.3 Student Submission Portal

- **REQ-5.1.3.1 (Mobile Responsiveness):** The Student Submission Portal shall be fully responsive,

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

supporting uploads from Mobile, Tablet, and Desktop devices.

- **REQ-5.1.3.2 (Receipt Display):** Since the system does not collect student email addresses, the portal shall display a **Digital Submission Receipt** on-screen immediately after a successful upload. This receipt acts as the only proof of submission.

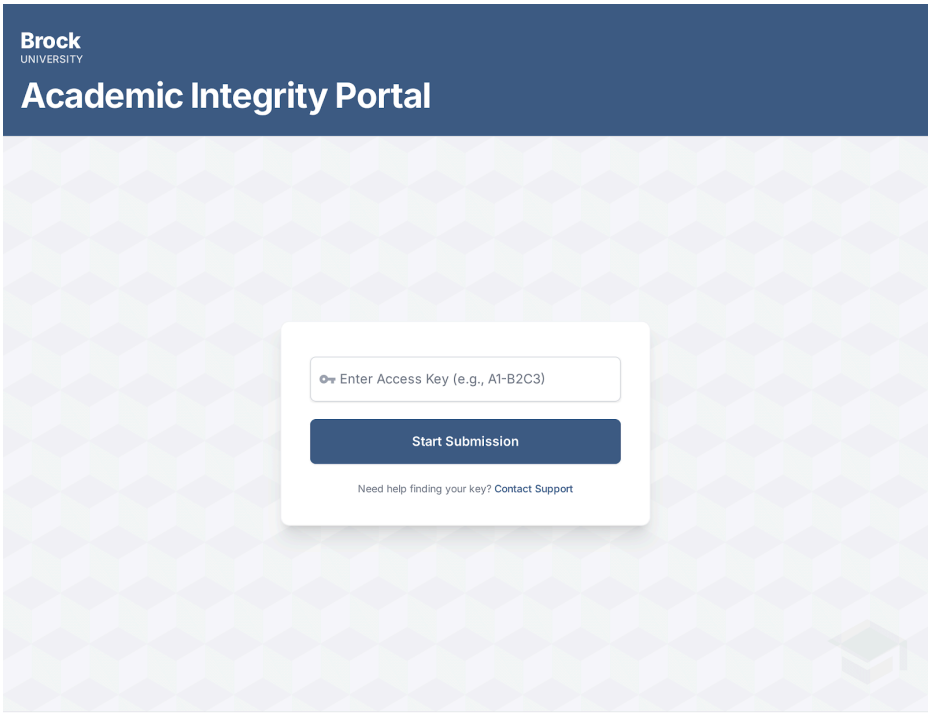


Figure 2: Student No-Login Access Interface

5.2 Software Interfaces

This section defines the internal and local communication protocols between the application backend and its supporting data layers.

5.2.1 Instructor Authentication

- **REQ-5.2.1.1:** The system shall interface with a local authentication database to validate Instructor credentials (Username/Password).
- **REQ-5.2.1.2:** The system shall **not** interface with external Single Sign-On (SSO) or LDAP systems (as per the standalone server requirement).

5.2.2 Database Interface

- **REQ-5.2.2.1:** The backend shall communicate with a Relational Database Management System

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

(RDBMS) to store assignment metadata, instructor profiles, and submission records.
- **REQ-5.2.2.2:** The backend shall communicate with a **File Object Storage** system to store and retrieve binary source code files (.java, .cpp, .zip).

5.3 Hardware Interfaces

- **REQ-5.3.1:** The system has no direct dependencies on specialized hardware devices (e.g., scanners, biometric sensors). It relies solely on the hosting server's standard processing and storage capabilities.

5.4 Communications Interfaces

This section specifies the protocols for data transmission and automated messaging within the system.

5.4.1 Network Security

- **REQ-5.4.1.1 (HTTPS):** All data transmission between the client browsers and the server shall be encrypted using **HTTPS (TLS 1.2 or higher)**. Unencrypted HTTP traffic shall be blocked or redirected.
- **REQ-5.4.1.2:** Student Metadata (Name/ID) shall be transmitted only over these encrypted channels.

5.4.2 Notifications

- **REQ-5.4.2.1: (Instructor Email):** The system shall be capable of sending automated email notifications to **Instructors** (e.g., "Analysis Batch Complete").
- **REQ-5.4.2.2: (No Student Email):** The system shall **not** send emails to students, as student contact information is not collected or stored. All student feedback must be immediate and on-screen.

6. Quality Attributes

This section specifies the non-functional requirements (NFRs) regarding how well the system performs its functions.

6.1 Usability

- **Priority: High**

- **REQ-6.1.1 (Performance Metrics):** The user interfaces (Instructor Dashboard and Student Portal) shall achieve a **Performance Score of 90 or higher** on Google Lighthouse / PageSpeed Insights tests for desktop devices.
    - **REQ-6.1.1.1 (Error Recovery):** The system shall provide clear, non-technical error

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

messages (e.g., "Invalid Access Key") to students, allowing them to recover from submission errors without instructor intervention.
    - **REQ-6.1.1.1 (Minimal Training):** The Student Submission workflow shall be designed to require no prior training, utilizing a linear, single-page wizard format
- **REQ-6.1.2:** Improved readability via merged match blocks and reduced visual noise.

.

6.2 Performance

- **Priority: Medium**

- **REQ-6.2.1 (Concurrency):** The system shall support a minimum of **50 concurrent student uploads** within a 60-second window (simulating a lab deadline rush) without request timeouts or service degradation.
- **REQ-6.2.2 (Analysis Speed):** The backend comparison engine shall process a batch of 50 active submissions against a repository of 500 archived files in under **5 minutes**.
- **REQ-6.2.3:** Efficient matching through reduced redundant hash comparisons.

6.3 Availability

- **Priority: Low**

- **REQ-6.3.1 (High Availability):** The system shall be designed for **99.9% Uptime** (allowing for approximately 8.7 hours of downtime per year). This requires the architecture to support redundant application instances or rapid container restarts.
- **REQ-6.3.2 (Graceful Degradation):** In the event of a Database connection failure, the Student Portal shall display a "Maintenance Mode" message rather than crashing with a raw stack trace.

6.4 Security

- **Priority: High**

- **REQ-6.4.1 (Input Validation):** The system shall sanitize all inputs (Student Name, ID, Access Key) to prevent SQL Injection (SQLi) and Cross-Site Scripting (XSS) attacks.
- **REQ-6.4.2 (No Encryption at Rest):** While data is encrypted in transit (HTTPS), source code files stored on the server disk are not required to be encrypted at rest, facilitating faster analysis access.
- **REQ-6.4.3 (Session Management):** Instructor sessions shall automatically time out after 60 minutes of inactivity to prevent unauthorized access on shared computers.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

6.5 Portability & Installability

- **Priority: Medium**

- **REQ-6.5.1 (Containerization):** The entire system (Backend API, Frontend, Database, Comparison Engine) shall be containerized using **Docker**.
- **REQ-6.5.2 (Deployment):** The system shall be deployable on any server environment (Cloud VM, On-Premises Linux Server) that supports a Docker runtime, satisfying the "housed on any server" project constraint.

7. Other Requirements

7.1 Legal and Regulatory Compliance

- **REQ-7.1.1 (Privacy Compliance):** The system shall comply with the **Freedom of Information and Protection of Privacy (FOIP)** Act by ensuring that no personally identifiable information (PII) is permanently stored beyond the active retention period.
- **REQ-7.1.2 (Data Anonymization):** The system shall strictly enforce the separation of student identity from submitted content, ensuring that "Outside Sources" (Guest Users) cannot access any personal data.

7.2 Internationalization and Localization

- **REQ-7.2.1 (Language):** The user interface (menus, buttons, error messages) shall be presented in **English** only.
- **REQ-7.2.2 (Date Format):** All timestamps (deadlines, submission times) shall be displayed in the ISO 8601 format (YYYY-MM-DD HH:MM) to avoid ambiguity between international users.

8. Glossary

This section defines the specialized terms, acronyms, and abbreviations used throughout this Software Requirements Specification.

8.1 Acronyms and Abbreviations

- **API:** Application Programming Interface.
- **FOIP:** Freedom of Information and Protection of Privacy Act.
- **HTTPS:** Hypertext Transfer Protocol Secure.
- **ISO:** International Organization for Standardization.
- **PII:** Personally Identifiable Information (e.g., Student Name, ID).

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

- **RDBMS:** Relational Database Management System.
- **SHA-256:** Secure Hash Algorithm 256-bit.
- **SQL:** Structured Query Language.
- **TLS:** Transport Layer Security.
- **UI/UX:** User Interface / User Experience.
- **UUID:** Universally Unique Identifier (128-bit label).
- **WCAG:** Web Content Accessibility Guidelines.
- **AODA:** Accessibility for Ontarians with Disabilities Act
- **FIPPA :** Freedom of Information and Protection of Privacy Act

8.2 Definitions

- **Access Key:** A shared, alphanumeric string generated by the system for a specific Assignment. It is distributed by the instructor to students off-platform and serves as the only authorization credential required to upload a submission.
- **Active Submission:** The most recent valid file uploaded by a student for an assignment. Only "Active" submissions are included in the similarity analysis. Previous drafts are marked as "Archived."
- **Anonymization:** The process of stripping PII (Name, Student ID) from a submission and replacing it with a random UUID to ensure privacy during storage and analysis.
- **Archived Submission:** A previous draft uploaded by a student that has been superseded by a newer upload. These files are stored for history but are excluded from current plagiarism checks.
- **Boilerplate (Template Code):** Code provided by the instructor (e.g., starter files, headers) that students are expected to include in their submission. This code is excluded from similarity calculations to prevent false positives.
- **Collusion Cluster:** A group of three or more submissions that share a high degree of similarity with each other, suggesting a coordinated effort to share solutions among a group of students.
- **Containerization:** The practice of packaging software (the system) with its dependencies (OS libraries) into a standardized unit (Docker Container) for development and deployment.
- **Encryption at Rest:** The security practice of encrypting data stored on the physical disk. (Note: This system explicitly *does not* require this for source code files, only for metadata in transit).
- **Guest User:** A user (typically a student) who interacts with the system without a permanent account or login credentials, identified only by the Access Key and session metadata.
- **Sanitization:** The automated process of parsing source code to remove comments, whitespace, and author tags (@author) before analysis.
- **Secondary Repository:** A collection of anonymized submissions from previous course offerings or external sources used as a reference database to detect plagiarism from past years.
- **Similarity Index:** A numeric value (0-100%) representing the percentage of code in a submission that matches text found in another submission or repository.
- **Stateless:** A system design where the server does not store session information between requests. The Student Submission Portal is stateless; each upload is an independent transaction validated by the Access Key.
- **Stimulus/Response:** A modeling technique used to describe the behavior of the system by defining an external event (Stimulus) and the system's specific reaction (Response).

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

- **Tokenization:** The process of breaking down source code into its basic functional elements (keywords, operators, literals) while ignoring non-functional text like comments and whitespace.
- **Winnowing:** An algorithm used to select a representative subset of hashes from a document. It makes the comparison process efficient and robust against small changes (like reordering code blocks).
- **Rolling Hash:** A method of calculating a hash value for a window of characters that "rolls" through the input, allowing for extremely fast generation of file fingerprints.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

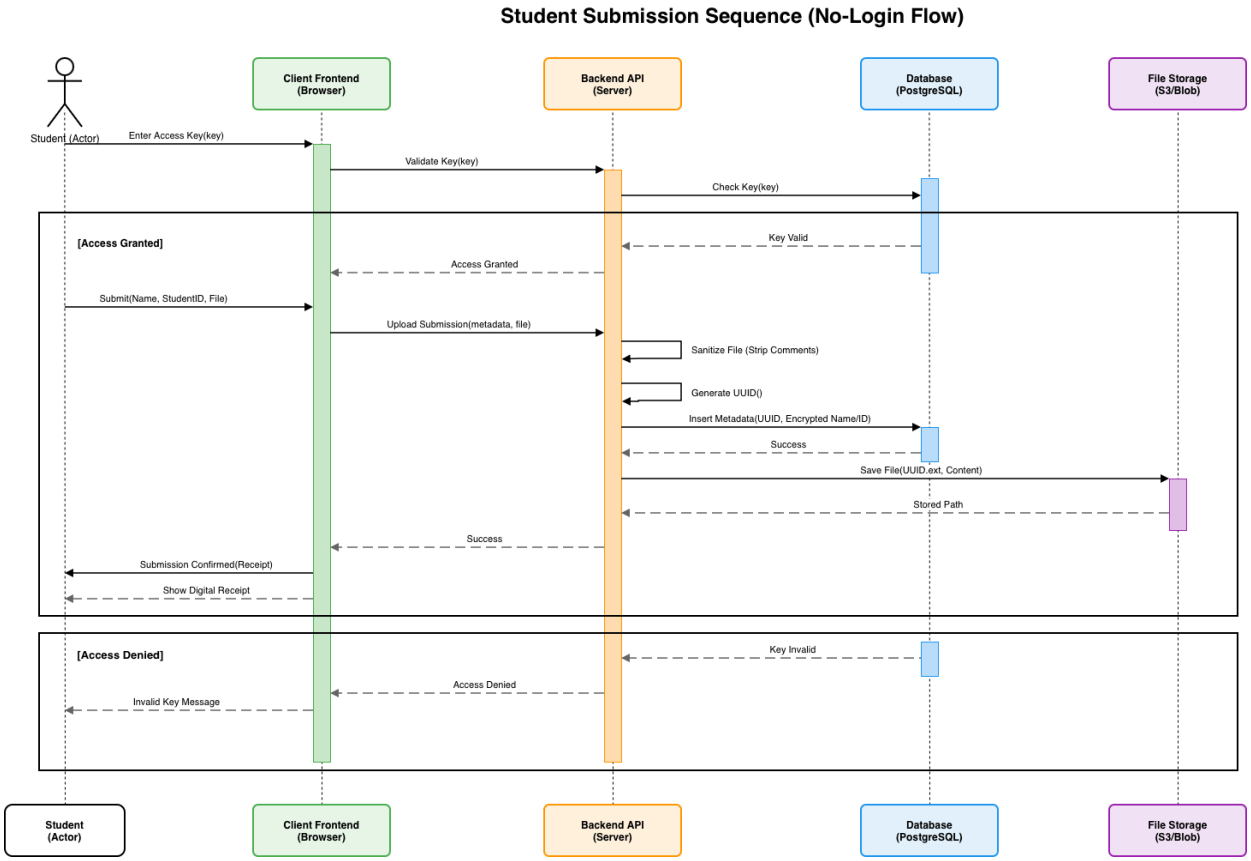
9. Analysis Models

This section provides visual models to elaborate on the system's functional requirements, data flows, and architectural design.

9.1 Sequence Diagrams

9.1.1 Student Submission Sequence (No-Login Flow)

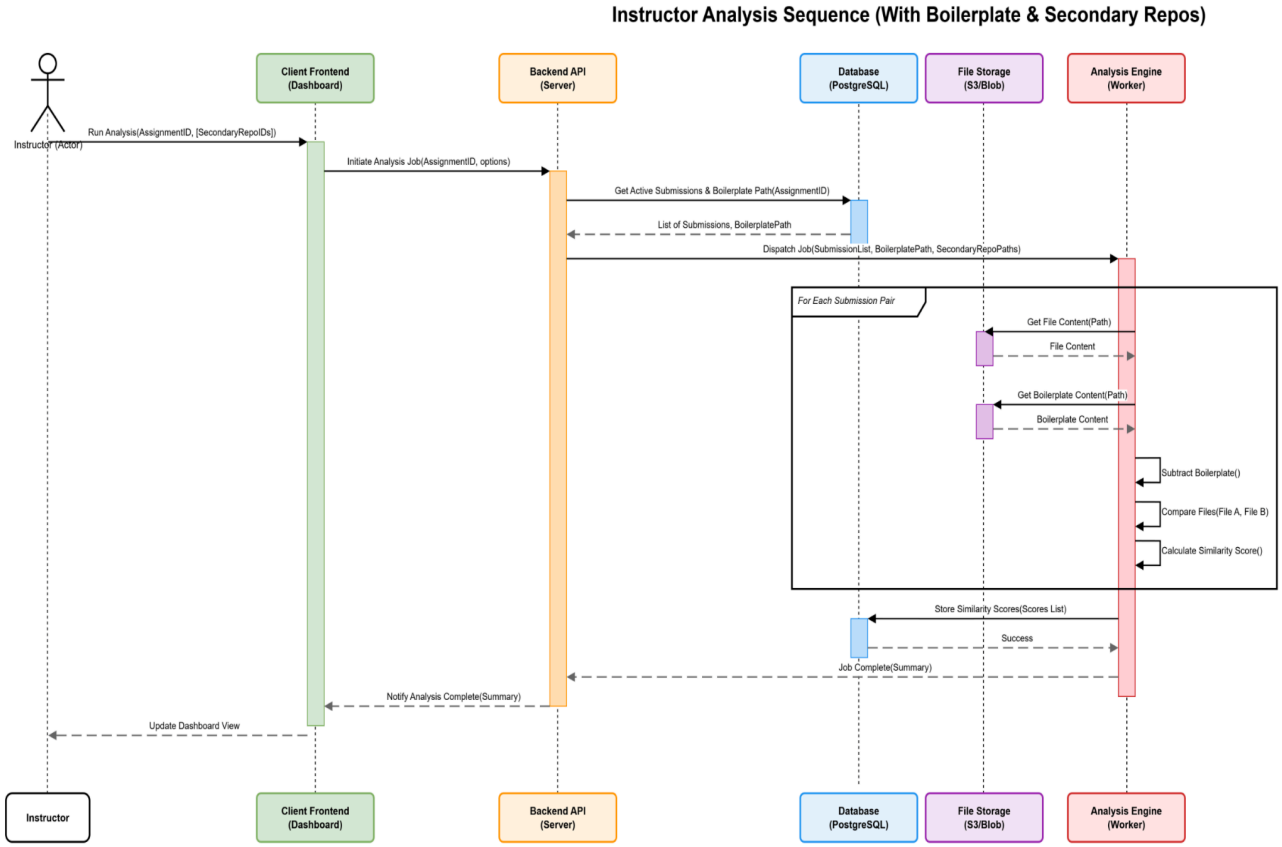
This diagram illustrates the stateless submission process, highlighting how a shared Access Key is used for validation and how student identity is immediately separated from the uploaded code file for privacy.



Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

9.1.2 Instructor Analysis Sequence

This diagram details the process of an instructor triggering a similarity analysis. It shows the comparison engine fetching active submissions, excluding boilerplate code, comparing files against both current and secondary repositories, and storing the resultant similarity scores.

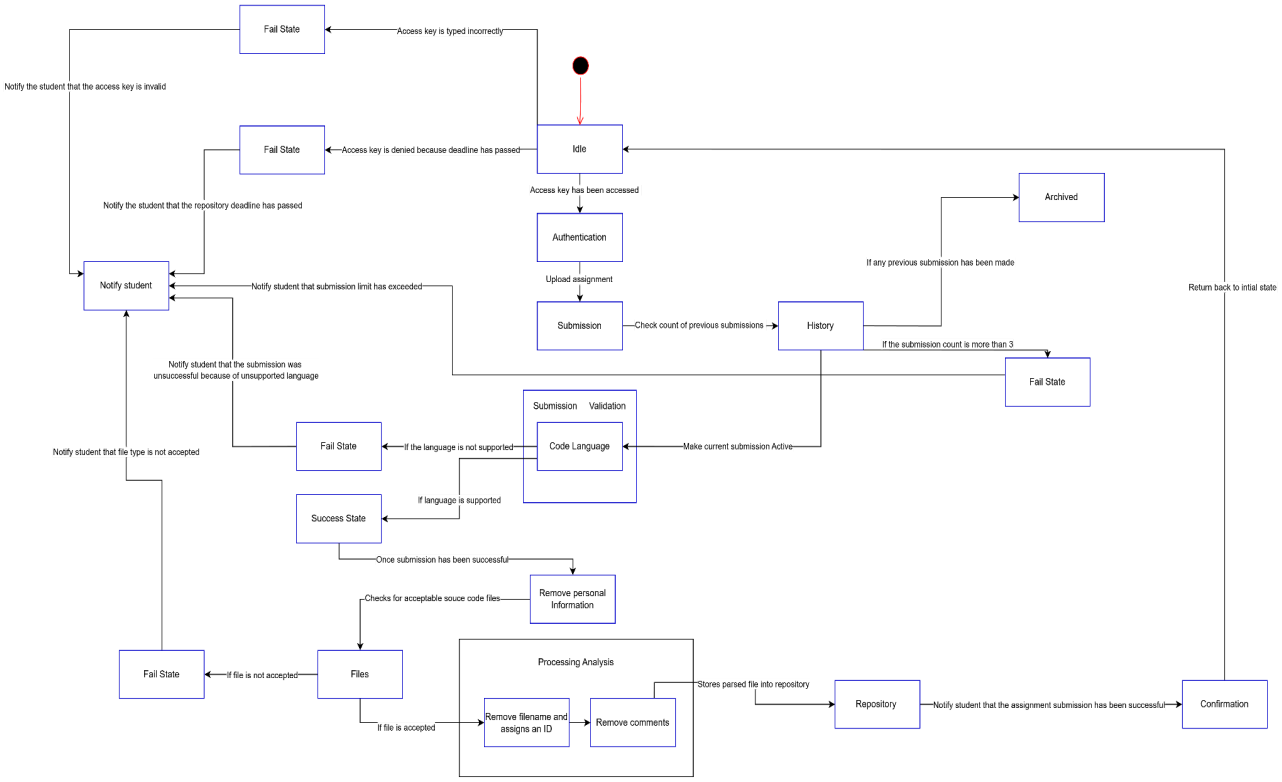


Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

10. State Machine Diagram

10.1 Submission Lifecycle State Machine

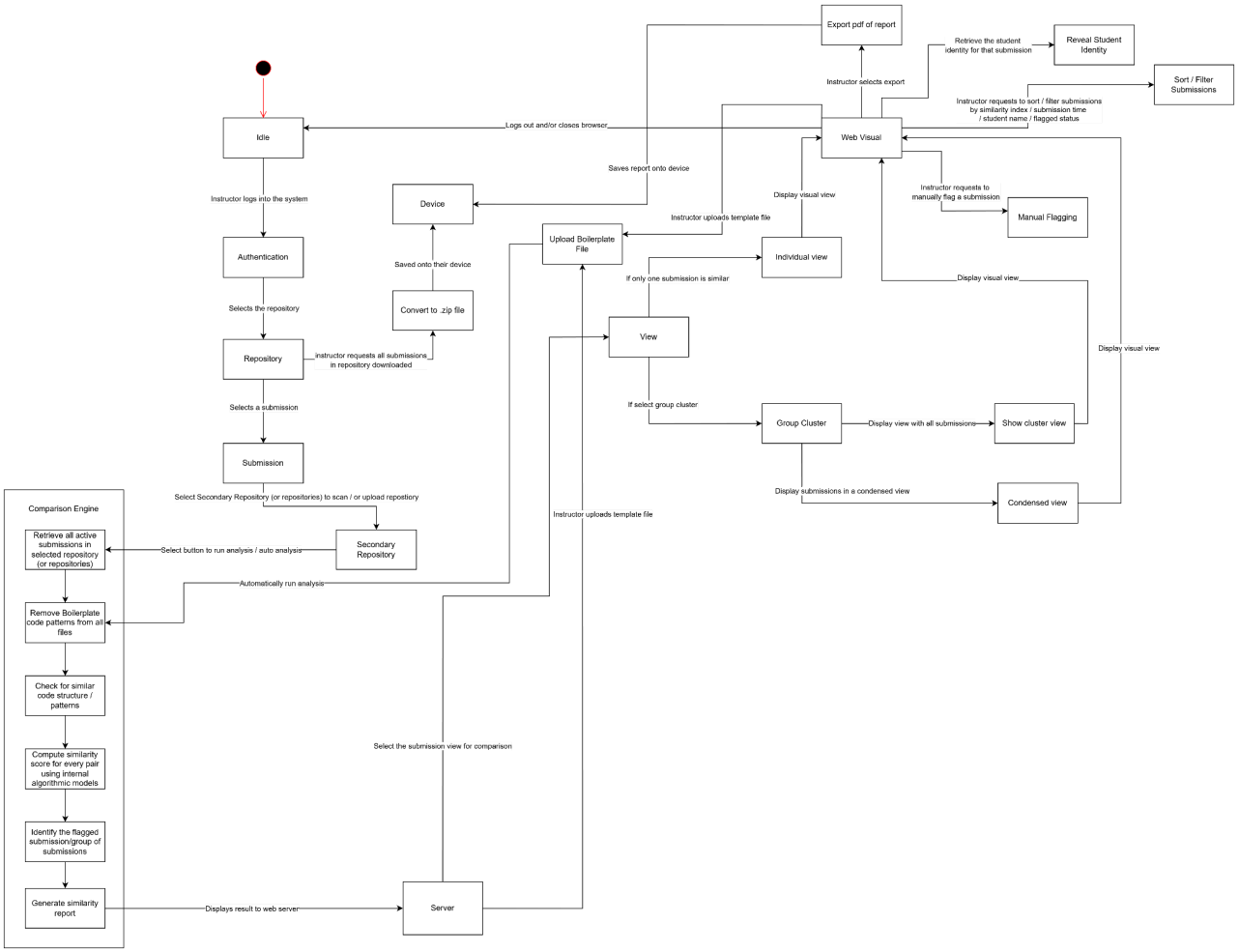
This model defines the various states of student submission:



Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

10.2 Analysis Lifecycle State Machine (Instructor)

This model defines the various states an instructor can inhabit, from analysing the similarity of code submitted by students.



Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

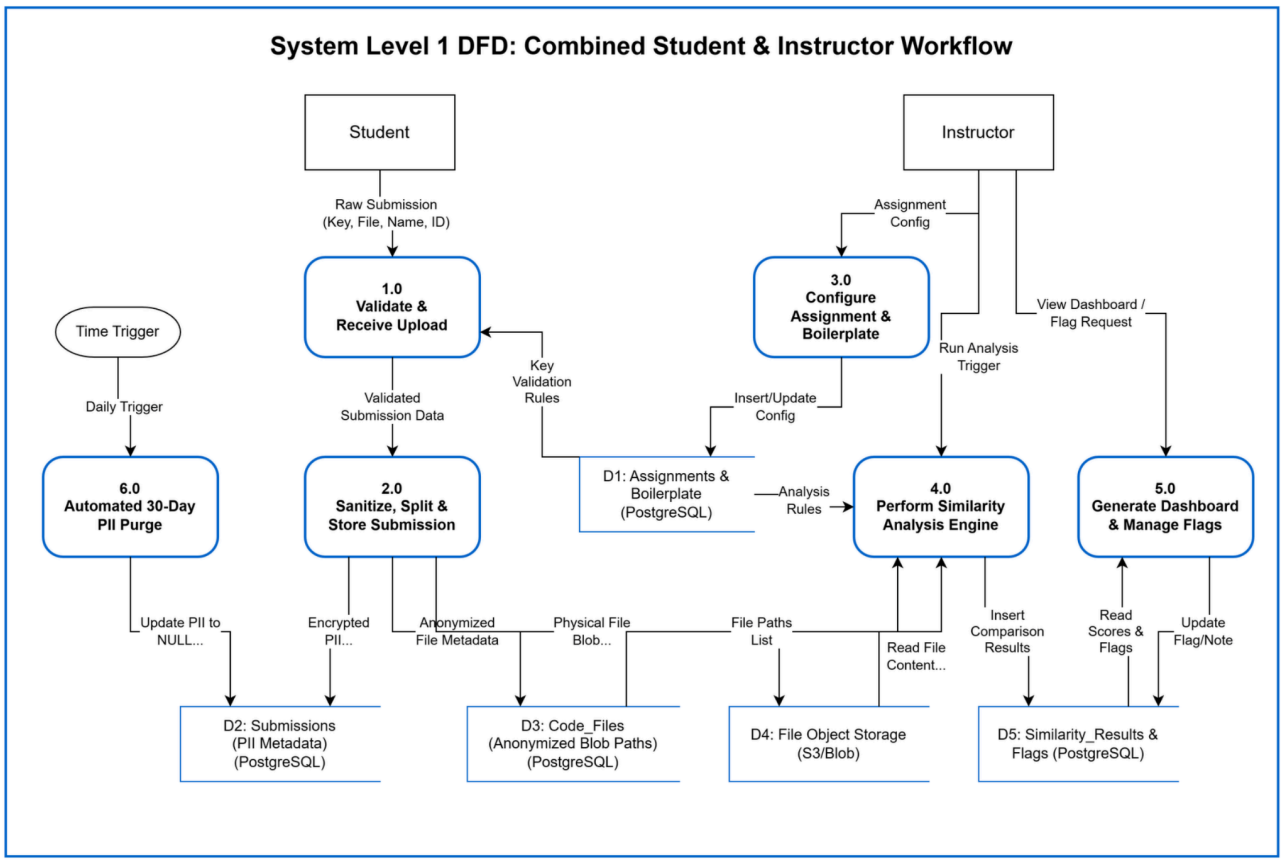
10.3 Data Flow Diagrams (DFD) - Level 1

10.3.1 Student Submission & Privacy Pipeline

This DFD focuses on the data ingestion process. It shows how a raw student submission is validated, sanitized, and split into separate storage locations for PII (Personally Identifiable Information) and anonymized code. It also illustrates the automated process that purges PII after 30 days.

10.3.2 Instructor Analysis & Dashboard Workflow

This DFD depicts the instructor's interaction with the system. It shows the data flow for configuring assignments, triggering the similarity analysis engine (which reads from multiple data stores), and generating the final dashboard view and reports based on the analysis results and flag status.

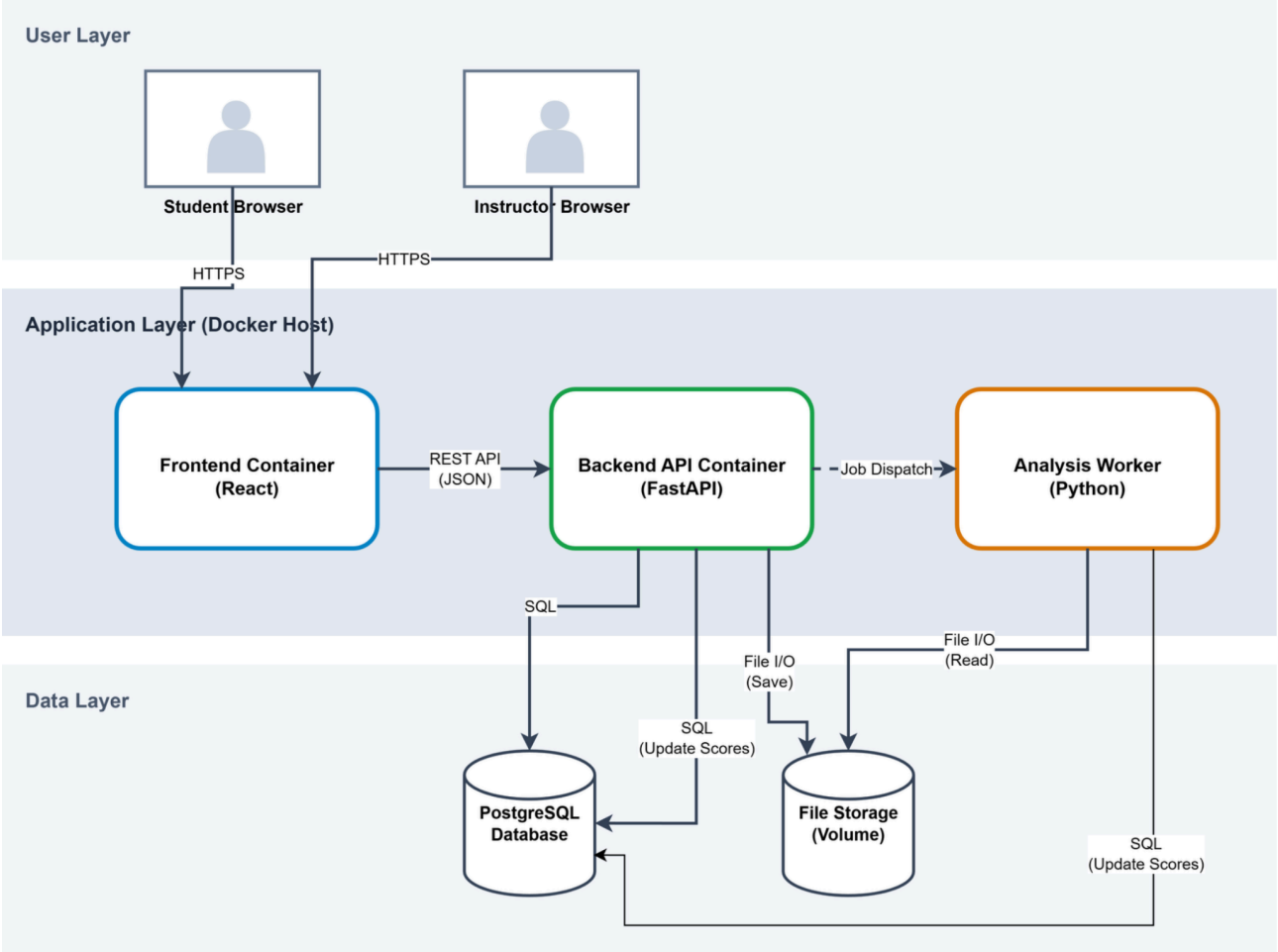


Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

10.4 System Architecture Diagram

10.4.1 Dockerized Microservices Architecture

This diagram provides a high-level view of the system's deployment architecture, showing the separation of concerns into Docker containers (Frontend, Backend API, Analysis Worker) and their interaction with the persistent data layer (Database and File Object Storage).



Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.